

# AI-based Village Pond Planning System — Technical Report

**Author:** Rahul Raj · **Course:** 7th semester, Assignment 1 · **Submission:** 5 September 2026 **Repository:** <https://github.com/Rahul5977/AI-BasedPondAnalysis> **Phase 2 route (deployed):** POST <http://10.1.75.53:4269/api/v1/analyzeContour> — running on the provided lab machine *stu78\_sys1*, campus network. Interactive documentation: <http://10.1.75.53:4269/docs> · web app: <http://10.1.75.53:4269/> . Also runnable anywhere: `make up` && `make seed` (full 11-service stack) or `make serve-single` (one process, no Docker); `make tunnel` adds a public ngrok URL for the demonstration.

## Abbreviations

| Abbreviation | Meaning                                                                    | Abbreviation | Meaning                                                       |
|--------------|----------------------------------------------------------------------------|--------------|---------------------------------------------------------------|
| ADR          | Architecture Decision Record                                               | JWT          | JSON Web Token                                                |
| AHP          | Analytic Hierarchy Process                                                 | KML / KMZ    | Keyhole Markup Language (KMZ: zipped)                         |
| AMC          | Antecedent Moisture Condition                                              | LLM          | Large Language Model                                          |
| API          | Application Programming Interface                                          | MCDA         | Multi-Criteria Decision Analysis                              |
| BFS          | Breadth-First Search                                                       | ML           | Machine Learning                                              |
| BoQ          | Bill of Quantities                                                         | NASA POWER   | NASA Prediction Of Worldwide Energy Resources                 |
| CN           | Curve Number                                                               | NDWI         | Normalised Difference Water Index                             |
| COG          | Cloud-Optimised GeoTIFF                                                    | OSM          | OpenStreetMap                                                 |
| CR           | Consistency Ratio (of the AHP matrix)                                      | RMS          | Root Mean Square                                              |
| CV           | Coefficient of Variation                                                   | RS256        | RSA signature with SHA-256 (JWT signing)                      |
| D8           | eight-direction flow routing — each cell drains to one of its 8 neighbours | SCS-CN       | Soil Conservation Service Curve Number (runoff method)        |
| DEM          | Digital Elevation Model                                                    | SHAP         | SHapley Additive exPlanations                                 |
| EAV          | Elevation–Area–Volume (curve)                                              | SPA          | Single-Page Application                                       |
| EPSG         | coordinate reference system code registry                                  | SRTM         | Shuttle Radar Topography Mission (the ~30 m elevation source) |
| ERA5-Land    | ECMWF Reanalysis v5, land component (rainfall dataset)                     | TIN          | Triangulated Irregular Network                                |
| ESA          | European Space Agency                                                      | TR-55        | USDA Technical Release 55 (urban hydrology handbook)          |
| FR           | Functional Requirement                                                     | TWI          | Topographic Wetness Index                                     |
| GIS          | Geographic Information System                                              | UI           | User Interface                                                |
| HSG          | Hydrologic Soil Group                                                      | UTM          | Universal Transverse Mercator (map projection)                |
| HTTP         | Hypertext Transfer Protocol                                                | VM           | Virtual Machine                                               |
| IMD          | India Meteorological Department                                            | XML          | Extensible Markup Language                                    |

## 1. Problem statement and objectives

Water conservation in rural India depends on small ponds built where the terrain will actually deliver runoff to them. Choosing such a site by hand needs a topographic survey, a rainfall record, a runoff estimate and a storage

calculation — expertise a Gram Panchayat rarely has on hand. The assignment asks for a web application that does this from geospatial and satellite data and presents the result on an interactive map (FR1–FR8).

The objective of this implementation is narrower and sharper than "a map with numbers": **every figure must be derived from the uploaded input, carry its unit and an honest uncertainty band, and be traceable to a named, cited algorithm** that the author can defend live. The examined topic — how the area is selected and how the catchment is computed — is therefore where most of the engineering depth sits (§4–§6).

## 2. Requirements coverage

| FR | Requirement                              | Where it is met                                                                                                                          | Evidence                                 |
|----|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1  | Satellite imagery for a selected village | Esri World Imagery basemap; village derived from the upload and named by reverse-geocoding its centroid; focus mask dims the outside     | docs/figures/p1-walking-skeleton.jpg     |
| 2  | Contour map visualisation                | Marching-squares contours from the working DEM, 1/2/5/10 m, labelled, Douglas-Peucker simplified (5 097 → 1 408 vertices at 2 m)         | p2-contours-5m.jpg                       |
| 3  | Land available for excavation            | Specification-pattern constraints on slope, water buffer (NDWI), habitation band, land cover, ownership, contiguity; parcels as polygons | p4-ndwi-opencv.png, available_land route |
| 4  | Catchment of a selected point            | Priority-Flood + D8 + nearest-channel snapping + upstream BFS; polygon, area, longest flow path, relief; validated                       | p2-click-to-catchment.jpg, §7            |
| 5  | Historical rainfall from public APIs     | Open-Meteo ERA5-Land (45 yr) with NASA POWER fallback; Weibull 75 % dependable, JJAS share, rainy days, Gumbel 25-yr                     | p3-rainfall-panel.jpg                    |
| 6  | Runoff volume                            | SCS-CN on the daily series, rational coefficient, Strange's table — a range with its spread                                              | p3-design-panel.jpg                      |
| 7  | Pond depth and storage                   | Cost-optimised excavated frustum, EAV curve, daily water balance → fill reliability, spillway, BoQ, confidence                           | p3-design-panel.jpg                      |
| 8  | All results overlaid                     | Results overlay with the six PDF items, pond footprint drawn at the outlet, 13 toggleable layers                                         | p5-all-overlays.jpg                      |

## 3. System architecture

### 3.1 Layers and services

The application is layered (ADR 0001) with dependencies pointing inwards only, enforced by `tests/test_layering.py`: `api` (routers, zero business logic) → `engines` (pure numpy algorithms and use-case workflows) → `providers` (external data adapters) / `repositories` (persistence) → `domain` (units, errors, value objects; imports nothing but numpy).

Eleven Docker Compose services, each arriving in the phase that first called it (ADR 0004): PostGIS, Redis, MinIO, the API, two Celery workers (interactive/heavy bulkheads), beat, TiTiler, nginx + the React/MapLibre SPA, Prometheus, Grafana.

### 3.2 Stack reconciliation against the suggested stack

| Suggested            | Chosen               | Why (ADR)                                                                                                                |
|----------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| Flask or FastAPI     | FastAPI              | Typed contract generated for free (OpenAPI, 40+ operations), async job routes, dependency injection for ports (ADR 0005) |
| MongoDB / PostgreSQL | PostgreSQL + PostGIS | Boundaries, <code>ST_Equals</code> village reuse, spatial types; append-only audit rules in the database (ADR 0006)      |

| Suggested                                   | Chosen                                                                                         | Why (ADR)                                                                                              |
|---------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Elevation APIs (point queries)              | Full DEM rasters from the uploaded contour map (and a documented provider-tile adapter)        | Catchment delineation needs a grid, not points; point APIs cannot give flow direction (ADR 0007, 0011) |
| Rainfall APIs (IMD, Open-Meteo, NASA POWER) | Open-Meteo ERA5-Land primary, NASA POWER fallback                                              | Keyless, daily, 45 years in 2 s; IMD gridded needs registration (decision log)                         |
| OpenCV                                      | OpenCV for the NDWI water mask: Otsu threshold, morphological open/close, connected components | Used where it is the right tool, not decoratively (ADR 0017)                                           |
| A basic frontend library                    | React + MapLibre GL                                                                            | Vector + raster layers, offline-first service worker (ADR 0018)                                        |

### 3.3 Design patterns applied

| Pattern                 | Where                                                                                                                       | Purpose                                                          |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Ports & Adapters        | <code>DEMProvider</code> , <code>RainfallProvider</code> , <code>ObjectStore</code> , <code>JobRunner</code> , repositories | Same pipeline in Docker, CI and tests; no mocks (ADR 0013)       |
| Strategy                | <code>RunoffMethod</code> (SCS-CN, rational, Strange); <code>DEMProvider</code> implementations                             | A range instead of one number; extensibility evidence (ADR 0010) |
| Specification           | <code>SlopeUnder</code> & <code>~WithinBuffer</code> & ...                                                                  | Readable, self-naming land constraints (ADR 0017)                |
| Builder                 | <code>PondDesignBuilder</code>                                                                                              | Stage-by-stage assembly of the FR7 payload (ADR 0016)            |
| Decorator               | <code>Retry</code> ◦ <code>CircuitBreaker</code> ◦ <code>Cached</code>                                                      | Resilience without touching the adapters                         |
| Chain of Responsibility | <code>FallbackChain</code> , ordered elevation strategies in the KML parser                                                 | First provider/strategy that answers wins; provenance recorded   |
| Flyweight               | Curve-number table keyed by (land cover, soil group)                                                                        | Eleven classes × four groups, not a raster of floats             |
| Observer                | WebSocket job progress over the job row                                                                                     | One source of truth, pushed on change                            |
| Saga                    | Persistence steps of the contour pipeline                                                                                   | Compensations in reverse on failure (ADR 0019)                   |
| State machine           | Recommendation lifecycle                                                                                                    | Illegal transitions are impossible, not merely unlikely          |
| Transactional outbox    | Recommendation events → audit log                                                                                           | Same-transaction event, asynchronous projection                  |
| Bulkhead                | Two Celery queues and worker pools                                                                                          | A heavy job cannot delay a click                                 |

## 4. Methodology

### 4.1 From contours to a DEM (P1)

The uploaded KML/KMZ is parsed tolerantly (namespace-agnostic, `<Folder>` root accepted) but elevation is read strictly, by an ordered strategy — Z coordinate, then a whitelisted `ExtendedData` field, then the placemark name — with numeric ID fields rejected, because the sample carries exactly such a decoy (ADR 0011). The UTM zone is derived from the file's centroid; `assert_crs()` refuses any computation in degrees.

Contour vertices are densified and triangulated (Delaunay TIN, linear interpolation), then lightly smoothed (Gaussian,  $\sigma = 1$  cell). The grid resolution is *derived*:  $\text{cell} = (\text{hull area} / \text{total contour length}) / 4$ , floored at the source DEM's resolution, which is inferred from the file's own metadata by an evidence-weighted match against a table of known datasets (the sample names SRTM five times and by raw filename; a generic blurb names Copernicus once). On the sample this gives a 30 m grid of  $110 \times 89$  cells, and the honest statement that a 1 m contour interval from a 30 m source is interpolated precision, not measured accuracy.

## 4.2 Hydrological conditioning and flow routing (P2)

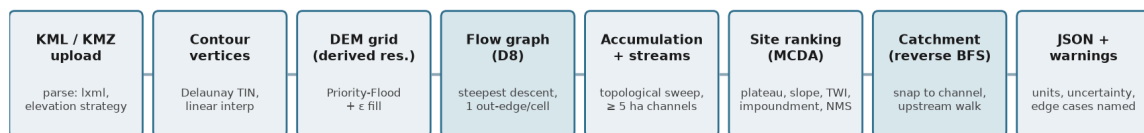
- **Depression filling and flat resolution:** Priority-Flood +  $\epsilon$  (Barnes, Lehman & Mulla 2014; Wang & Liu 2006). One pass,  $O(n \log n)$ ; the fill-depth raster is itself a figure (p2-sink-fill-before-after.png: 2 141 cells, max 8.6 m — TIN hollows and a river floor below its map-edge exit).
- **Flow direction:** D8 (O'Callaghan & Mark 1984), steepest descent over eight neighbours, diagonals divided by  $\sqrt{2}$  (ADR 0009: deterministic, the GIS default, defensible;  $D-\infty$  is a documented stub).
- **Flow accumulation:** one descending-elevation pass — on a filled+ $\epsilon$  surface that order is topological, so a single sorted loop is exact. Pure numpy, no numba.
- **Streams:** cells with  $\geq 5$  ha upstream (a threshold in area, so it means the same on any grid), traced into links with Strahler (1957) order; on the sample 101 links, order 3, 24.4 km, visibly following the Sivinath and its tributaries (p2-streams-on-satellite.jpg).
- **Derived surfaces:** Horn (1981) slope/aspect/hillshade; Zevenbergen & Thorne (1987) profile and plan curvature (ArcGIS sign convention); TWI =  $\ln(a / \tan \beta)$  (Beven & Kirkby 1979).

## 4.3 Catchment delineation (FR4)

A click one cell off a channel returns a hillslope catchment orders of magnitude too small, so the pour point is snapped to the **nearest** cell within 150 m that drains  $\geq 2$  ha — nearest, not largest, so a tributary click is not dragged onto the main river — and the distance moved is returned and shown. The catchment is the breadth-first search over the inverse D8 graph from the snapped outlet; area, perimeter, longest flow path (BFS distances), relief and outlet elevation follow. A catchment reaching the map edge is flagged `catchment_truncated`. Uncertainty on the area is stated as  $\max(15\%, \text{one cell ring around the perimeter})$  — on a 38 ha catchment of 30 m cells that is  $\pm 26\%$ .

## 4.4 The whole computation as a graph algorithm

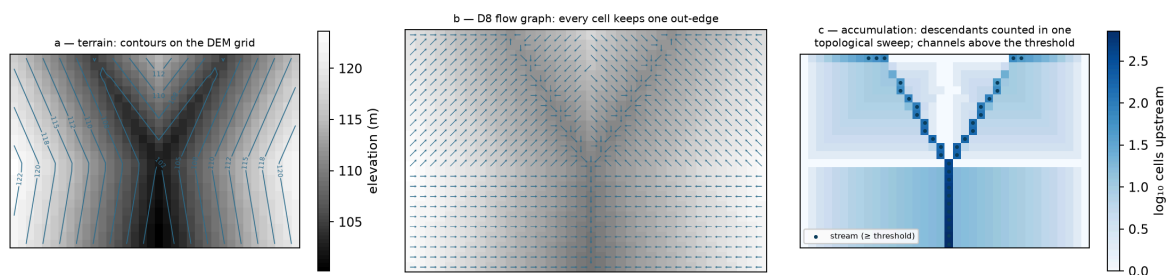
The pipeline's core is a single graph structure built once per upload and reused by every downstream engine. The figures below are generated from the real engine code on a small synthetic Y-valley (`make figures`), so they cannot drift from the implementation.



Highlighted stages are the graph algorithm: the D8 flow graph and the reverse breadth-first search over it.

The `analyzeContour` pipeline; the highlighted stages are the graph algorithm

**Nodes and edges.** Every DEM cell is a node. D8 gives each node **at most one out-edge** — to the steepest-descending of its eight neighbours (panel b below). A graph in which every node has out-degree  $\leq 1$  is a *functional graph*; because the surface has been filled and given an  $\epsilon$ -gradient it contains no cycles, so it is a **forest of in-trees rooted at the outlets** (edge cells where water leaves the map). Everything the API returns is a question about this forest:

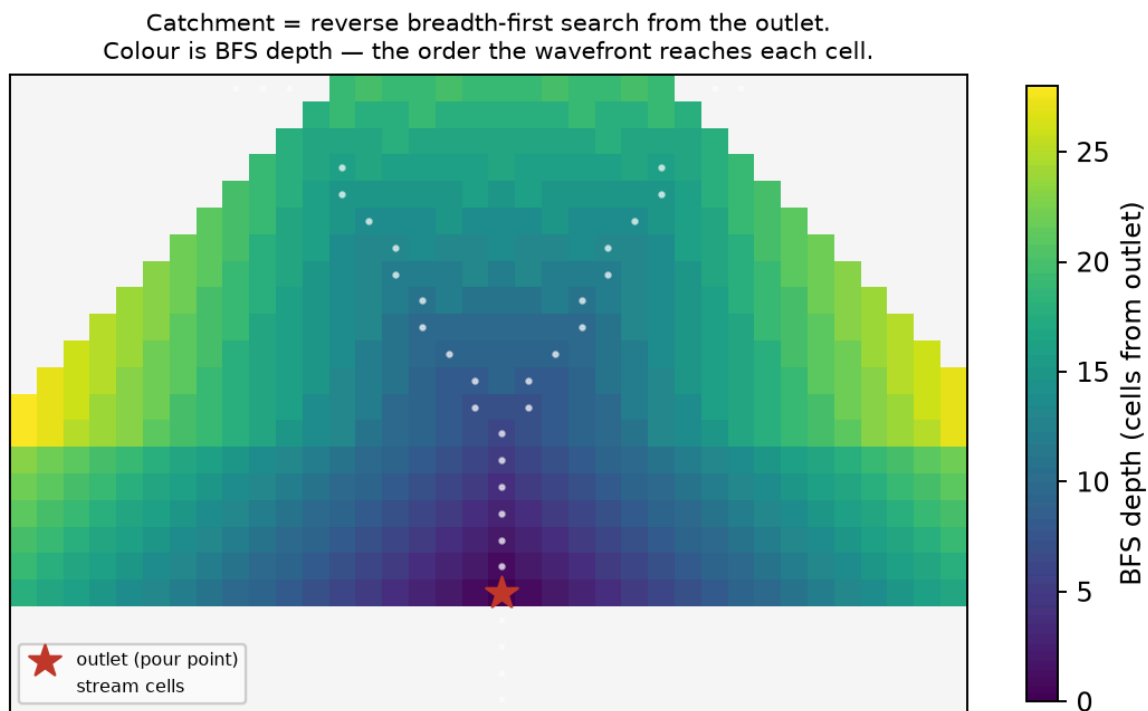


DEM → D8 flow graph → accumulation and streams

- **Flow accumulation = subtree size.** The number of cells draining through a node is the size of the subtree rooted at it. Rather than recursing, the engine visits nodes in descending elevation — on a filled +  $\epsilon$  surface

that order is a **topological order** of the graph — and pushes each node's count onto its receiver: one exact  $O(n)$  sweep (panel c).

- **Streams = a thresholded subgraph.** Channel cells are nodes whose subtree exceeds 5 ha (a threshold in area, so it means the same at any grid resolution). Walking the stream subgraph from sources to junctions yields links and Strahler order — order rises only where two children of equal order meet, a property of the tree, not of geometry.
- **Catchment = reverse breadth-first search.** The catchment of a pour point is the set of nodes from which the outlet is reachable: a BFS over the *inverted* edges, exactly the textbook traversal, shown as its wavefront below. Area, longest flow path (the deepest BFS level  $\times$  cell size, diagonals  $\sqrt{2}$ ) and relief fall out of the same walk.



Catchment as a reverse BFS from the outlet; colour is the wavefront order

**Complexity and measured cost.** Priority-Flood fill  $O(n \log n)$ ; the elevation sort  $O(n \log n)$ ; accumulation, stream extraction and each BFS  $O(n)$ . The sample's  $110 \times 89$  grid (9 790 nodes) completes the whole pipeline in  $\sim 2$  s inside the request worker; the catchment BFS itself runs in about a millisecond. Nothing here needs numba or a GPU at village scale — a deliberate choice, because every line must be explainable live.

#### 4.5 Site selection (Phase 2 "where"; ADR 0014)

Candidates are drainage-network cells not within three cells of the map edge, with slope  $\leq 15\%$ . Each is scored by a weighted sum of four memberships in  $[0, 1]$ :

- **upstream area** — a plateau in  $\log_{10}$  hectares: 0 at 1 ha, 1 from 10 to 150 ha, 0 at 1 000 ha. A first version scored area monotonically and put every top site on the main river; a village pond is not a river dam;
- **flatness** — 1 on 0–3 % slope, falling to 0 at 15 % (an optimum at exactly 0 % would reward floodplains and DEM artefacts);
- **wetness** — normalised TWI;
- **impoundment efficiency** — the volume held behind a nominal 2 m rise (8-connected flood fill on the upstream side of a bund) divided by the pool footprint: a mean depth that rewards natural basins.

Weights come from a declared Saaty pairwise matrix by the principal eigenvector (AHP, Saaty 1980); the consistency ratio is computed with Saaty's random index and returned — CR = 0.004 for the default matrix; an intransitive matrix is rejected in a golden test. Non-maximum suppression at 200 m keeps the top-N distinct. The top site's catchment

is delineated and returned with the ranking and every candidate's per-criterion scores, so the answer to "why here?" is in the response.

#### 4.6 Rainfall (FR5)

Daily precipitation from Open-Meteo's ERA5-Land archive (1981–2025 for the sample's centroid), NASA POWER as fallback, both behind `Retry` ◦ `CircuitBreaker` ◦ `Cached` and a `FallbackChain` that records which provider answered. Statistics on complete calendar years only (incomplete years excluded, never scaled): mean, median, CV, **75 % dependable rainfall by the Weibull plotting position**  $P = m/(n+1)$  (the design figure in Indian minor-irrigation practice), June–September share, IMD rainy days ( $\geq 2.5$  mm), maximum 1-day, monthly normals, and the 25-year 1-day depth by a Gumbel EV1 fit (method of moments) on annual maxima.

#### 4.7 Runoff (FR6)

Land cover from ESA WorldCover 2021 (10 m), read as a window straight from the public COG; topsoil texture from ISRIC SoilGrids → hydrologic soil group (USDA rule of thumb; default C with a warning when SoilGrids times out). The composite curve number is the area-weighted TR-55 value over the catchment (AMC II, Hawkins adjustment available). Three methods run on the daily series and report 75 % dependable annual runoff volumes:

- **SCS-CN** (USDA-SCS 1972; TR-55 1986):  $Q = (P - I_a)^2 / (P - I_a + S)$ ,  $S = 25400/CN - 254$  mm,  $I_a = 0.2 S$ , per day, then summed — applied to an annual total it overestimates runoff more than threefold (asserted by a test);
- **runoff coefficient** (rational form, ASCE coefficients by land cover): the crude upper bracket;
- **Strange's table** (1928, Madras PWD) by daily rainfall and catchment condition.

The spread between them is reported, not hidden (188 % on the sample's top site); SCS-CN is the design figure.

#### 4.8 Pond design (FR7; ADR 0016)

Target storage = 75 % dependable runoff  $\times$  0.6 harvest efficiency, clamped to [2 000, 50 000] m<sup>3</sup> (a village pond, not a reservoir; when the cap binds the response says so). Geometry: an excavated inverted frustum, 2:1 side slopes, prismatic volume, 0.5 m freeboard, bottom  $\geq 5$  m each way. **Depth is derived by cost:** a grid search over depth 1.5–3.5 m and aspect 1–2 solves each candidate's top dimensions for the target and picks the cheapest (₹160/m<sup>3</sup> cut, ₹220/m<sup>3</sup> bund, indicative 2024 bands), ties to the smaller water surface. A daily water balance over the record (inflow = SCS-CN runoff  $\times$  area  $\times$  0.6; evaporation 0.7  $\times$  pan from a monthly climatology; seepage 2 mm/day; 15 % dead storage) yields the fill reliability — the share of years reaching  $\geq 90$  % — plus months with water and mean spill. The spillway is sized for the 25-year storm (IMD short-duration reduction, Kirpich time of concentration, rational peak, broad-crested weir). A bill of quantities and a confidence label decided by the worst input (assumed land cover or soil, cached rainfall, edge-limited catchment) complete the payload; the natural EAV curve of the site is reported alongside.

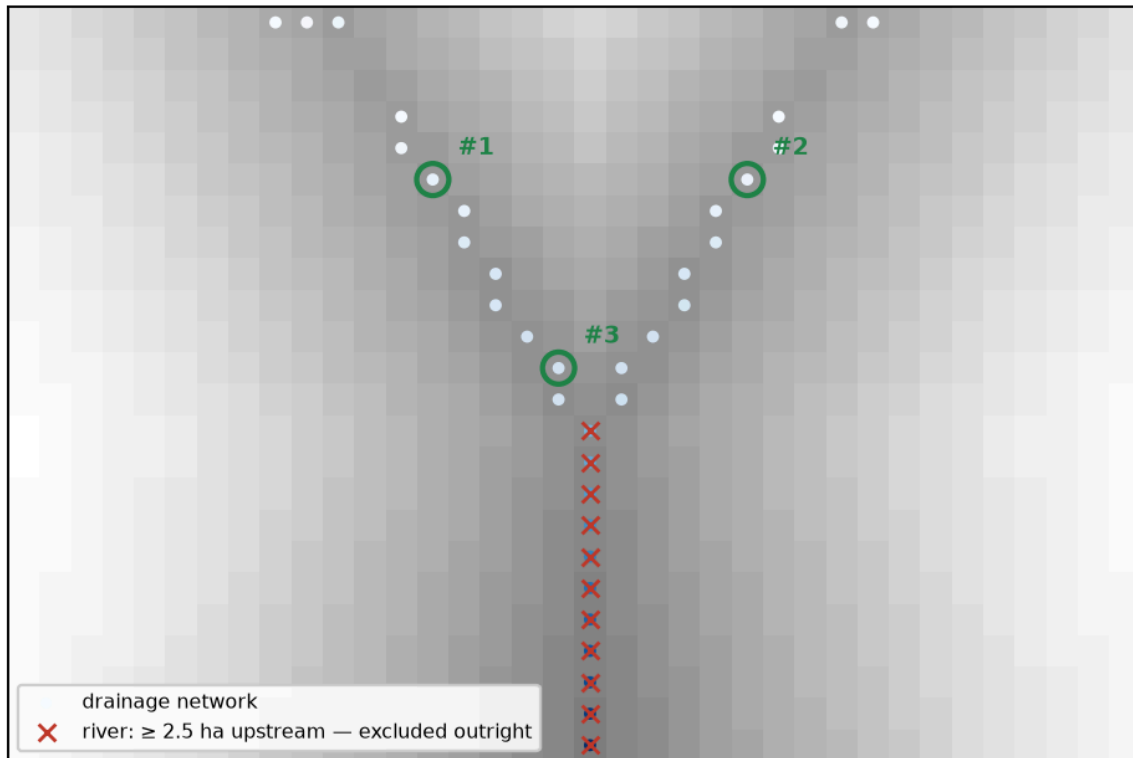
#### 4.9 Edge cases, and what the API says about them

An analysis that silently produces a wrong-looking number is worse than one that refuses; the edge cases below are handled *and named in the response* so the caller knows what happened.

| Edge case                                      | Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                              | Where it is proven                                                                                                |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>A river already crosses the area</b>        | Channel cells at or beyond the plateau's upper bound (1 000 ha default) are hard-excluded from siting — with no better cell available a soft score alone would still put the pond on the river. Any channel beyond the 10–150 ha ideal triggers the <code>existing_watercourse</code> warning naming the largest channel's size. On the sample the response says the ~416 ha watercourse is avoided and candidates sit on tributaries. | golden test<br><code>test_an_existing_river_is_excluded_from_siting_not_just_scored_down</code> ;<br>figure below |
| Flat / featureless terrain                     | No drainage network forms → siting returns an empty candidate list, the route answers with the <code>no_site_found</code> critical warning rather than an arbitrary point                                                                                                                                                                                                                                                              | golden test <code>test_flat_terrain_yields_no_candidates...</code>                                                |
| Catchment cut by the map edge                  | The uploaded extent bounds the analysis, not a real divide: <code>catchment_truncated</code> warning; candidate sites keep a 3-cell margin from the edge so their catchments are not trivially cut                                                                                                                                                                                                                                     | engine + API tests; visible on the sample                                                                         |
| Garbage / empty / non-XML upload               | 422 <code>unsupported_input</code> problem document, never a 500                                                                                                                                                                                                                                                                                                                                                                       | parser tests; e2e check                                                                                           |
| Elevation decoy ( ID field), missing elevation | Ordered elevation strategy with a whitelist; a file with only an ID-like numeric field raises <code>elevation_not_found</code> rather than guessing                                                                                                                                                                                                                                                                                    | parser tests on the sample's own decoy                                                                            |
| Click far from any channel                     | Snap searches 150 m for a cell draining $\geq 2$ ha; nothing found → 422 validation problem naming the radius                                                                                                                                                                                                                                                                                                                          | <code>test_snap_moves_a_flank_click...</code>                                                                     |
| Depressions / closed basins in the TIN         | Priority-Flood fills them (2 141 cells on the sample, max 8.6 m) and the fill depth is reported as a figure, not hidden                                                                                                                                                                                                                                                                                                                | <code>p2-sink-fill-before-after.png</code>                                                                        |
| Provider outages (rainfall, soil, land cover)  | Fallback chain → cache → stated defaults, each with a warning and a lowered confidence label; the demo runs fully offline                                                                                                                                                                                                                                                                                                              | resilience tests with injected failures                                                                           |

| Edge case            | Behaviour                                                                                 | Where it is proven                 |
|----------------------|-------------------------------------------------------------------------------------------|------------------------------------|
| Geocoder unreachable | Village named from its coordinates with the <code>geocode_unavailable</code> info warning | seen live on the lab-VM deployment |

Edge case: an existing watercourse. Channel cells beyond the plateau's upper bound are a river — hard-excluded; candidates land on tributaries and are reported.



The existing-river edge case: river cells hard-excluded, candidates on tributaries

#### 4.10 Demonstration on the provided contour map

The Phase 2 route, run against the deployed URL with the provided sample (data/samples/contours\_1m.kml , 6.4 MB, 1 355 contour lines):

```
curl -s -F "file=@data/samples/contours_1m.kml" \
  http://10.1.75.53:4269/api/v1/analyzeContour      # → 202 {"job_id": ...}
curl -s http://10.1.75.53:4269/api/v1/jobs/$JOB      # → "succeeded" in ~8 s
curl -s http://10.1.75.53:4269/api/v1/analysis/results/contour/$JOB
```

The response (trimmed to the headline fields; captured live from the URL above):

```
{
  "village_name": "Khapri",
  "contour_count": 1355,
  "utm_epsg": 32644,
  "suggested_pond_location": {"lon": 81.28415, "lat": 21.26222},
  "location_rationale": "Highest composite score (0.93) of 672 drainage cells:
    upstream area 38.2 ha, local slope 1.1 %, TWI 14.0, impounds 255,031 m3
    behind a 2 m rise (mean depth 1.50 m).",
  "catchment": {
    "area": "38.25 ha ( $\pm 26$  %)",
    "longest_flow_path": "902.13 m ( $\pm 10$  %)",
    "relief": "13.00 m ( $\pm 65$  %)"
  },
  "candidate_sites": 5,
```

```

"siting": {
  "weights": {"upstream_area": 0.35, "flatness": 0.2,
    "wetness": 0.15, "impoundment": 0.3},
  "candidates_considered": 672,
  "max_upstream_area_ha": 415.7
},
"warnings": ["interpolated_precision", "existing_watercourse", "catchment_truncated"]
}

```

Everything above is derived from the upload: the village name by reverse-geocoding the file's own centroid, the UTM zone from its coordinates, the 30 m working grid from its contour spacing and stated SRTM provenance, the pond location and catchment from the flow graph of §4.4. The same map in the browser — upload, layers, click-to-catchment, results — is shown in the Appendix figures, and the full payload for every route is captured in `docs/api/samples/`.

## 5. Software design and quality

- **Code quality:** `ruff` clean, `mypy --strict` on `domain/` and `engines/`, 206 tests, layering enforced by an executable test; every engine module's docstring names the algorithm and its citation. Coverage: **engines 94.3 %**, **domain 97.6 %**, **overall 86.2 %** (`docs/figures/p7-coverage.jpg`).
- **Test taxonomy:** golden tests with analytic answers (inclined plane, cone, V-valley, Y-valley, artificial pit, flat terrace, frustum hand calculation, SCS-CN hand calculation, monotonicity of runoff in rainfall), parser tests on the sample and on `Z/ExtendedData/KMZ/` decoy variants, recorded-response tests for the rainfall adapters, resilience-decorator tests with injected failures, end-to-end API flows on the real sample with the in-memory adapters, the pysheds cross-check, and the saga compensation test.
- **Git hygiene:** conventional commits, one feature per commit, no secrets or generated binaries (COGs are regenerated by `make seed`), `.env.example` documents every variable.

## 6. System design and management (ADR 0019)

Async jobs with `202 + poll/WebSocket` and real stage percentages; two bulkhead queues (a catchment returned in 0.6 s while a suitability job was running); idempotency keys; backpressure `429 + Retry-After`; RS256 JWT with viewer/planner/officer roles; a recommendation state machine whose transitions write a transactional outbox drained into an append-only audit log; request-correlation ids in every JSON log line; Prometheus metrics from the API and both workers with a provisioned Grafana dashboard; a leader-elected nightly rainfall refresh; an offline-first service worker (ADR 0018). Load test (Locust, 50 users, 60 s): 1 102 catchment submissions, POST p95 33 ms, end-to-end p95 560 ms, 0 HTTP failures. Chaos test: with the API container stopped, the reloaded page still shows the village, its layers, rainfall and sites from cache under an "Offline" badge (`docs/media/chaos-test.gif`); the test caught a real bug on its first run — a reverse proxy answers 502, which is a response, not a network error — now handled.

Scaling arithmetic and the distributed roadmap are in `docs/PLAN.md` Part 9; the bottleneck to national deployment is cadastral data and institutional trust, not compute.

### 6.1 Deployment on the provided lab machines

The provided machines (`student@10.1.75.53`, four systems) turn out to be unprivileged containers: no Docker daemon, no systemd, no root. This is exactly the situation the ports-and-adapters decision (ADR 0013) was made for — the same engines run behind in-memory persistence, an inline job runner and a local object store, so the whole system deploys as **one uvicorn process** serving both the API and the built SPA (`scripts/single_server.py`, `make serve-single`):

- **URL:** `http://10.1.75.53:4269` (host port 4269 → container port 4000 on `stu78_sys1`); Swagger at `/docs`, the planner at `/app`.
- **Install:** copy the tree, `uv sync --no-dev`, `~/pond/run.sh` (recorded-rainfall mode, so no outbound dependency); restart is the same script.

- **Verified:** the same 42-check end-to-end suite passes against the lab VM as against the full Docker stack (`docs/figures/e2e-api-lab-vm.txt`), including a live Sentinel-2 suitability read from the lab network.
- **Trade-off, stated:** raster tile layers need TiTiler and are absent in this mode; every vector product (catchment, contours, streams, sites) is served by the API and renders normally. The full 11-service stack remains the reference deployment (`make up`).

The readiness probe originally reported this deployment as permanently degraded because it probed postgres and redis unconditionally; it now probes only the *configured* adapters — a real bug found by deploying, fixed with a test.

Deploying the same codebase two ways — eleven services with bulkheads, backpressure and observability on Docker, and a single process on a bare student VM — is the practical scalability lesson of this project: the layering pays for itself when the environment changes, and the load numbers above say when the heavier shape is actually needed.

## 7. Validation and results

| Check                             | Method                                                                                 | Result                                                                                                                                                                                                       |
|-----------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interpolation                     | Inclined plane and cone contours, analytic elevations                                  | RMS < 0.15 m (plane), < 0.25 m between rings (cone); summits above the top contour are flattened — a documented limitation                                                                                   |
| Sink filling                      | Synthetic pit; flat terrace                                                            | Pit filled exactly to its spill level, nothing else touched; every cell drains after +ε                                                                                                                      |
| Accumulation                      | Inclined plane (count of upslope cells); bowl                                          | Exact match; bowl drains to its centre unfilled and to its rim filled                                                                                                                                        |
| Delineation                       | V-valley: catchment = the rectangle upstream of the outlet; Y-valley: Strahler order 2 | Exact                                                                                                                                                                                                        |
| <b>Independent implementation</b> | pysheds (Bartos 2020) on the sample DEM, outlets snapped in both models                | 2.0 %, 3.4 % (Jaccard 0.98, 0.97) on the main outlets; 22.5 % (0.77) on a floodplain flat where the two flat-resolution schemes route differently — reported, not hidden (ADR 0015)                          |
| Pour-point sensitivity            | Catchment area over a ±3-cell window around a channel cell                             | CV 212 % without snapping, 48 % with ( <code>p2-pour-point-sensitivity.png</code> )                                                                                                                          |
| Stream threshold                  | Overlay on imagery                                                                     | The 5 ha network reproduces the Sivnath and its main tributaries without hillside noise                                                                                                                      |
| Water mask                        | Sentinel-2 NDWI, three post-monsoon scenes                                             | 32 raw components → 20 water bodies, 9.0 % of the area; WorldCover 2021 independently says 8.1 %                                                                                                             |
| AHP                               | Default matrix; perfectly consistent matrix; intransitive matrix                       | CR 0.004; 0.000; rejected (> 0.10)                                                                                                                                                                           |
| Rainfall                          | Recorded ERA5-Land, Khapri                                                             | 45 complete years; mean 1 313 mm, 75 % dependable 1 168 mm, monsoon share 89 %, 86 rainy days                                                                                                                |
| Runoff                            | Hand calculation, CN 80, 50 mm day                                                     | Q = 13.8 mm; annual-total shortcut > 3× the daily sum                                                                                                                                                        |
| Design                            | Frustum hand calculation; optimiser feasibility; water balance on synthetic seasons    | Exact; cheapest feasible design chosen; small pond fills every year, oversized pond never                                                                                                                    |
| <b>Existing ponds</b>             | OSM's two mapped tanks inside the extent, designed at their centroids                  | Local runoff is a tenth of their capacity (canal-fed tanks in the Durg command area); natural impoundment at +2 m within −40 % of the larger tank's capacity ( <code>p3-existing-pond-comparison.md</code> ) |
| Load                              | Locust 50 users / 60 s                                                                 | POST p95 33 ms, end-to-end p95 560 ms                                                                                                                                                                        |

| Check                | Method                                                                                                                                                                                                                   | Result                                                                                                               |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| End-to-end API smoke | make e2e — 42 checks over HTTP: every real route in cookbook order plus the negative paths (viewer approve → 403, draft → approved → 409, missing village → 404, garbage upload → 422, the existing_watercourse warning) | 42/42 against the Docker stack and 42/42 against the lab-VM deployment (e2e-api-local-stack.txt, e2e-api-lab-vm.txt) |

Sample-map headline numbers (Khapri, Durg, Chhattisgarh, EPSG:32644, 830 ha): top site drains 38.25 ha ( $\pm 26\%$ ); 75 % dependable rainfall 1 168 mm; CN 88 (HSG C); SCS-CN 99 k m<sup>3</sup> (rational 244 k, Strange 18 k); design 50 000 m<sup>3</sup> (capped; harvestable 59.5 k), 3.5 m deep, 126 × 126 m top, fills in 100 % of years, ₹97 lakh indicative, confidence *low* (soil assumed); 27 ha of eligible land in 16 patches.

## 8. API documentation

The API is documented in four forms, all generated from or verified against the running code:

| Artifact                                       | Where                                                 | What it gives                                                                                                     |
|------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Interactive documentation (Swagger UI / ReDoc) | http://10.1.75.53:4269/docs · /redoc (any deployment) | Every route, schema and example, try-it-out enabled                                                               |
| Machine-readable contract                      | docs/api/openapi.json (regenerate: make openapi)      | The full OpenAPI 3.1 document — 40+ operations; the frontend's types are generated from it                        |
| Cookbook                                       | docs/api/cookbook.md                                  | Every workflow as copy-paste curl, with real captured responses in docs/api/samples/                              |
| Error catalogue                                | docs/api/errors.md · live at GET /api/v1/meta/errors  | Every stable error code with its HTTP status — generated from the same table the handlers use, so it cannot drift |

Conventions: long analyses are jobs (202 + job\_id, poll GET /jobs/{id} or the WebSocket); every number is a quantity object with value, unit, uncertainty band and method; errors are RFC 9457 problem documents with stable codes; fixture-backed routes (only parcel import) label themselves with X-Fixture-Data: true.

## 9. Limitations and uncertainty

The DEM is interpolated from contours that are themselves interpolated from ~30 m SRTM ( $\pm 6$  m relative): relief below about 5 m is not real, single-cell extremes are lost, and the sample's summit and valley floor sit 2–3 m inside the contour range. Every storage figure therefore carries  $\pm 20\%$  or worse and is planning-grade, not survey-grade. Ownership is unknown without a cadastral layer and is never assumed. Reanalysis rainfall is  $\pm 15\%$  against gauges. The three runoff methods disagree by up to 2×; the design uses SCS-CN and says so. SoilGrids is slow and the default soil group is a stated assumption. The ML scoring path is deferred by the plan's own fallback: two mapped tanks are not a training set (ADR 0017).

## 10. Future work

Ground-truth against 20–30 existing ponds in one block; a cadastral import with ownership class only (DPDP Act 2023); local CN calibration; the provider-DEM adapter (Copernicus GLO-30) for villages without a contour map; the ML/SHAP path once labels exist; the distributed roadmap in docs/PLAN.md Part 9.

## 11. Use of AI tools

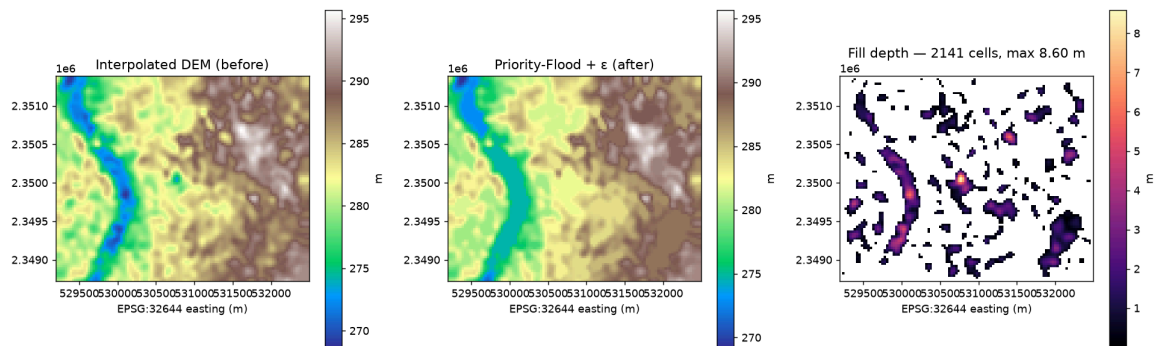
An AI coding assistant (Claude Code) was used, under the author's direction and review, as the assignment's LLM policy permits. It was used for:

1. **Code refactoring** — refactoring passes over the author-directed implementation: extracting the workflow orchestrators, the ports-and-adapters split, and tightening the layering that the architecture test enforces;

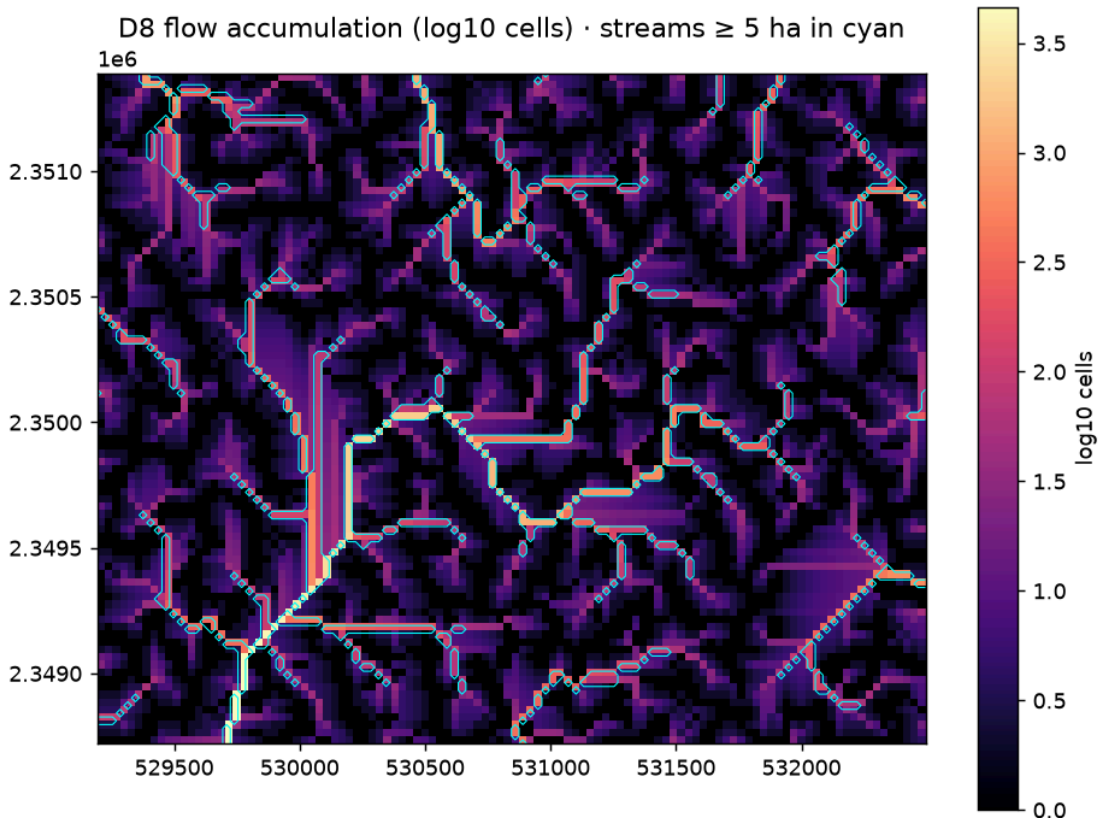
2. **Brainstorming to understand the algorithm (D8)** — working through D8 flow routing as a functional graph, why a filled +  $\epsilon$  surface makes descending elevation a topological order, why catchment delineation is exactly a reverse breadth-first search, and what the edge cases are (an existing river in the area, flat terrain, truncated catchments) — the understanding written down in §4.4 and §4.9;
3. **Writing structured code** — keeping the code organised in the layered structure of §3.1 (routers free of business logic, pure engines, adapters behind ports), with typed contracts, docstrings that name each algorithm, and tests alongside every engine;
4. **Enhancing the report** — editing this document, generating the explainer figures (`scripts/make_algorithm_figures.py`), and improving its structure and presentation; every number in it is produced by the code, not by the assistant.

Every design decision is recorded with its reasoning and rejected alternatives in `docs/adr/` (19 records) and the decision log in `docs/PROGRESS.md`; every algorithm is named and cited in its module docstring; the author can explain and justify each component live.

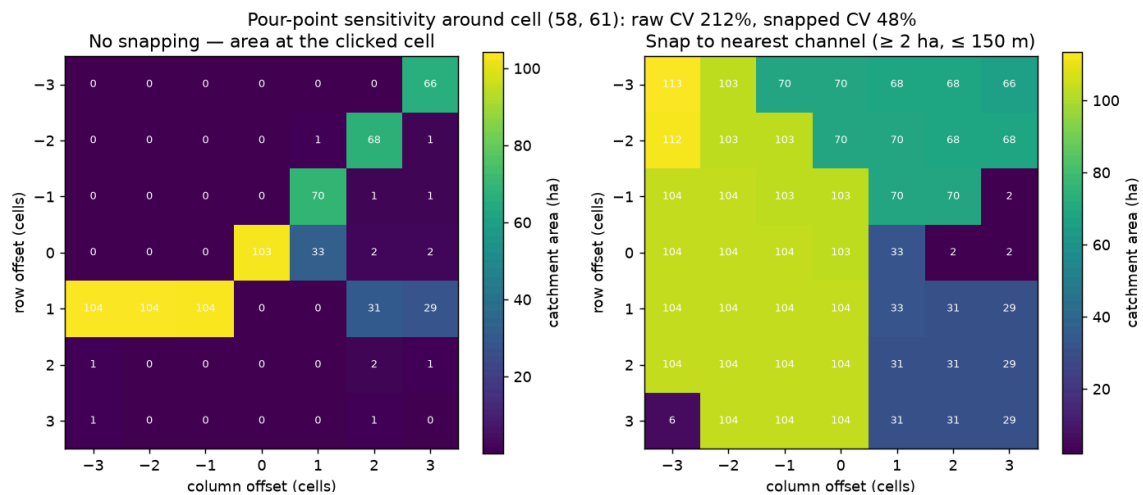
## Appendix A — Figures



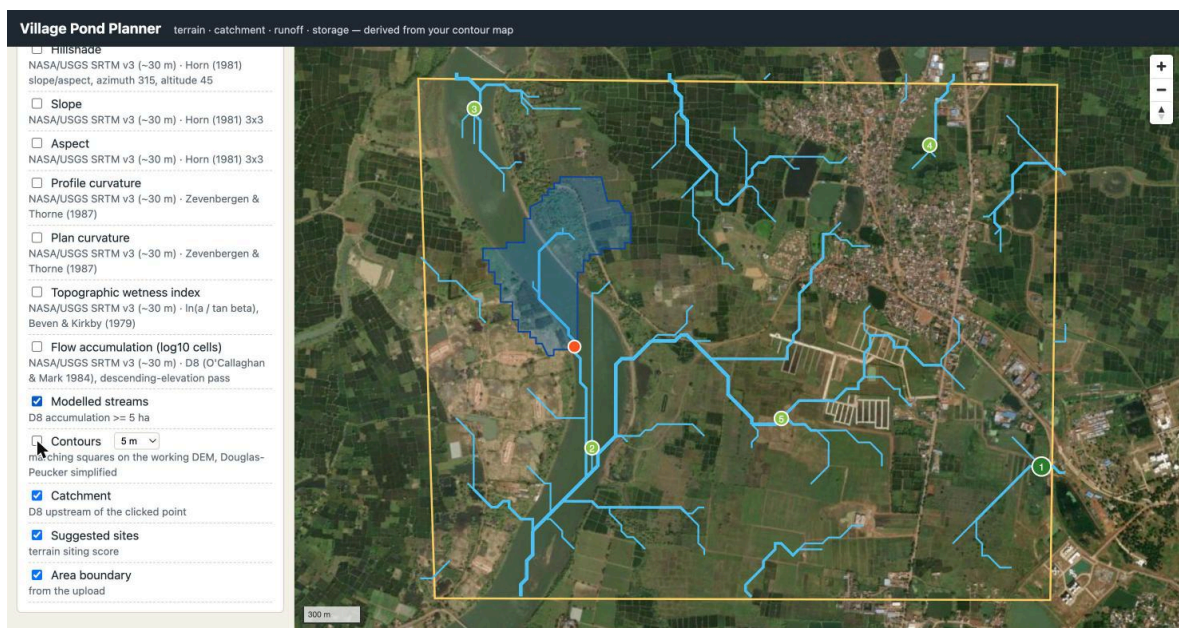
Sink filling on the sample DEM: 2 141 cells filled, maximum 8.6 m, before/after



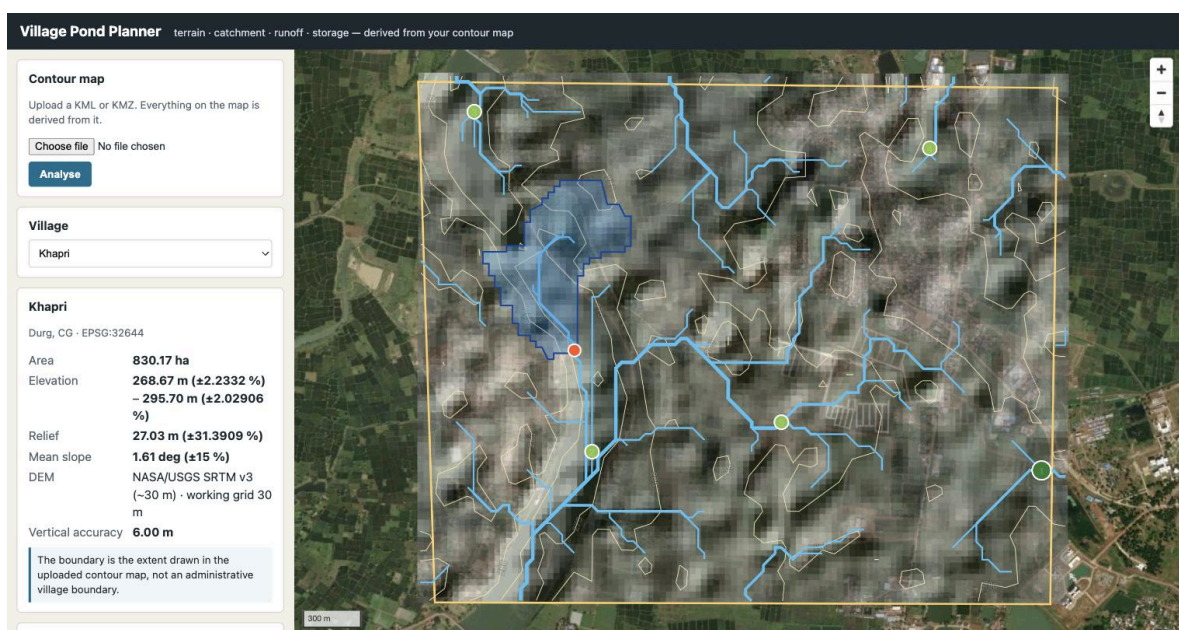
D8 flow accumulation (log scale) with the 5 ha stream network



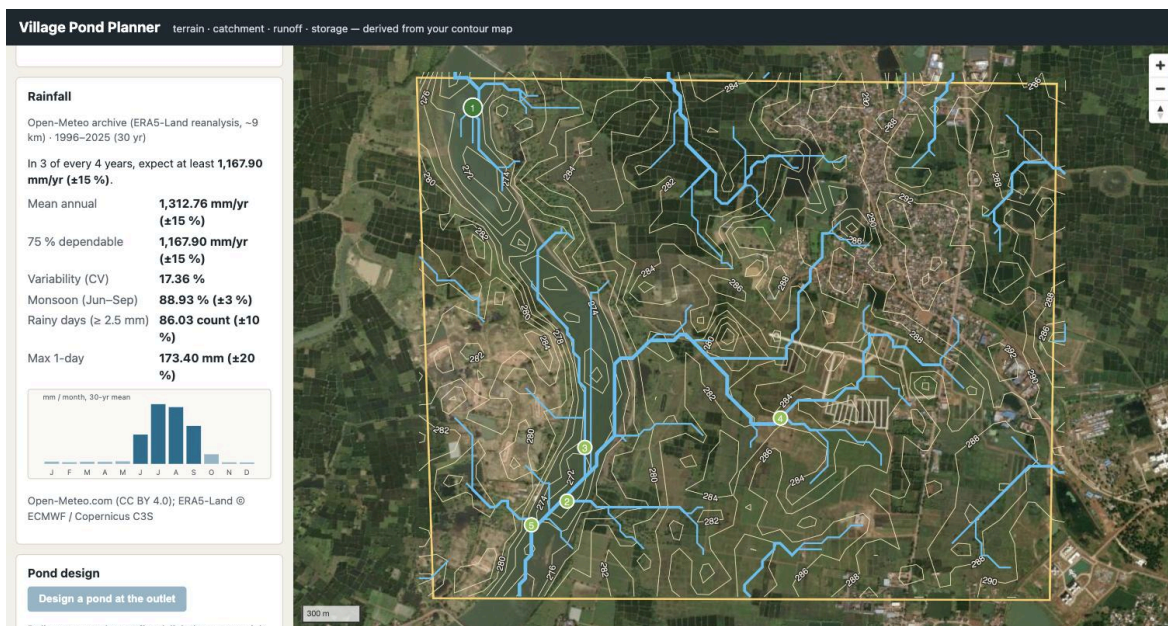
Pour-point sensitivity: catchment area around a channel cell, raw vs snapped



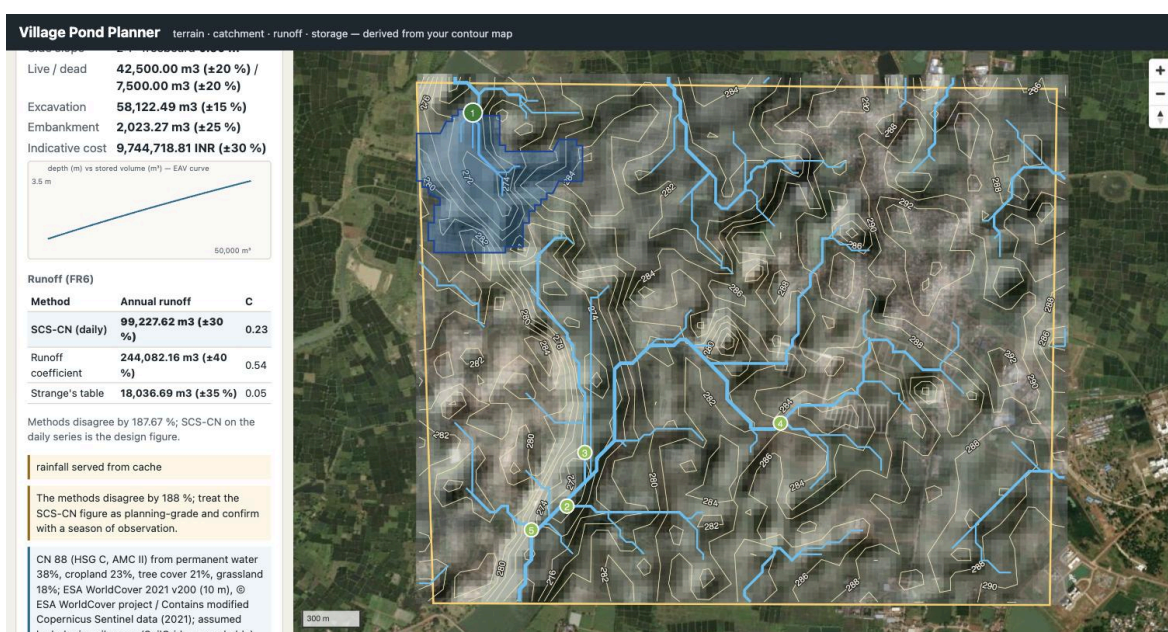
Modelled streams over satellite imagery — the Sivnath and its tributaries



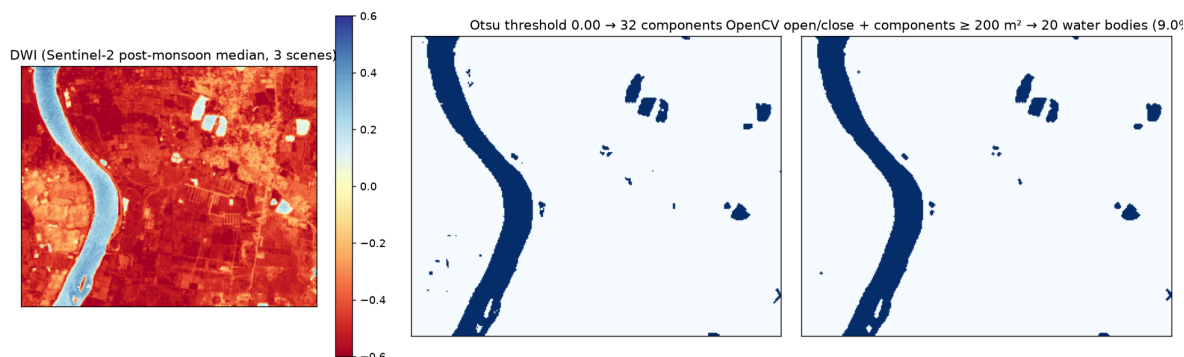
Click-to-catchment in the browser: snapped outlet, snap distance, polygon, area with its band



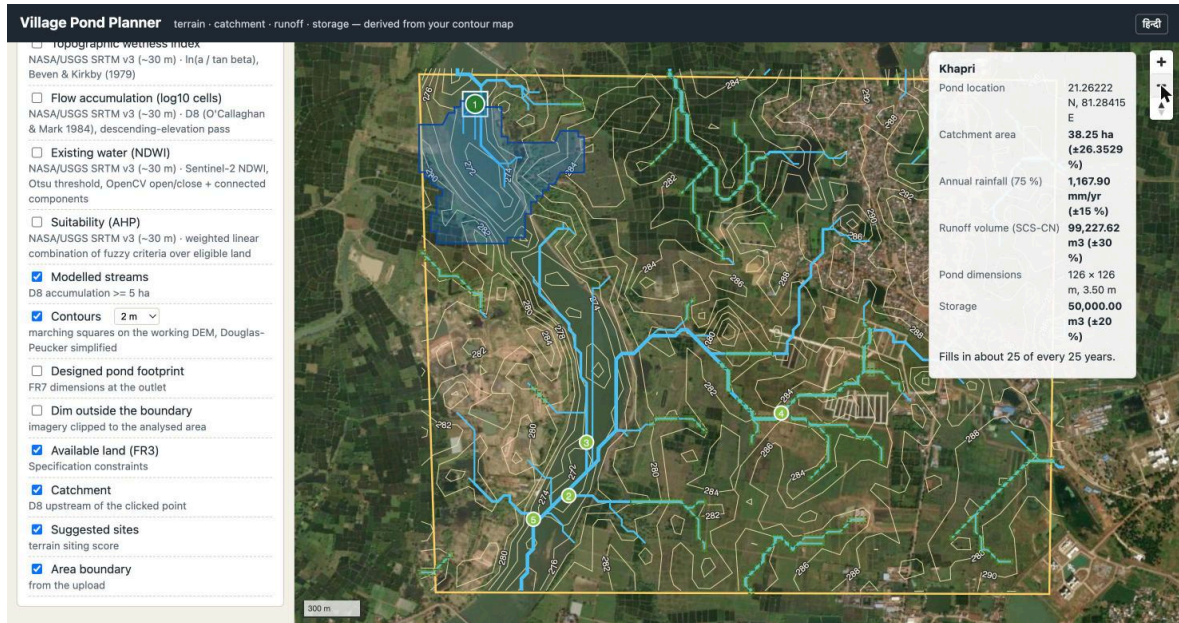
Rainfall panel: 45 years of ERA5-Land, 75 % dependable year, monthly normals



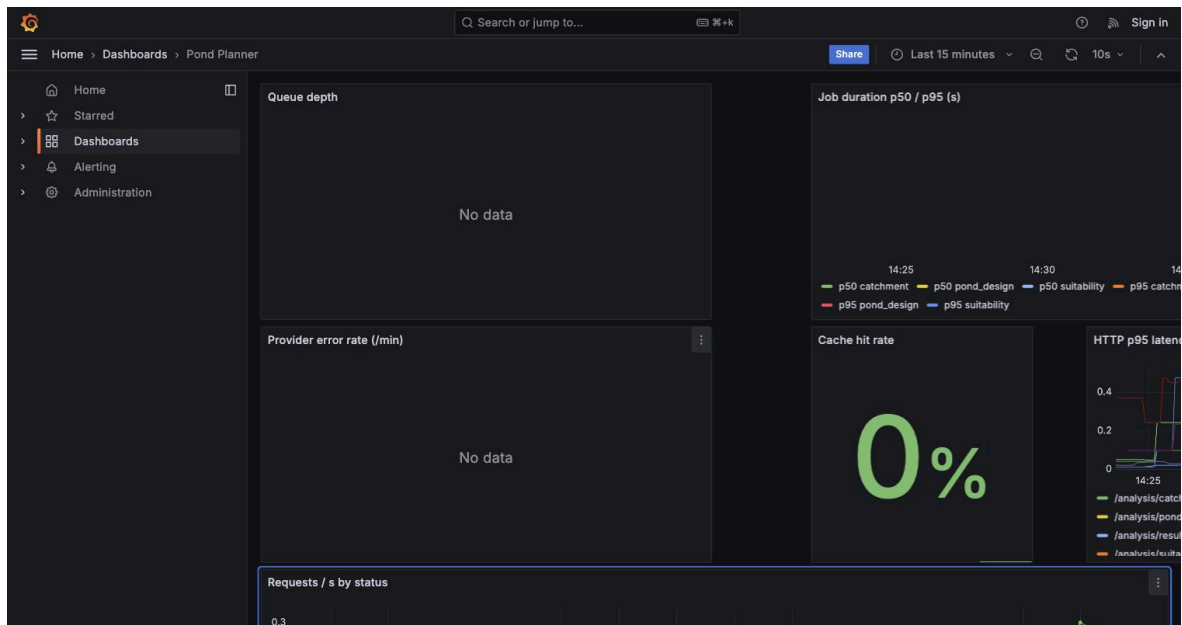
Pond design panel: dimensions, EAV curve, three runoff methods, fill reliability



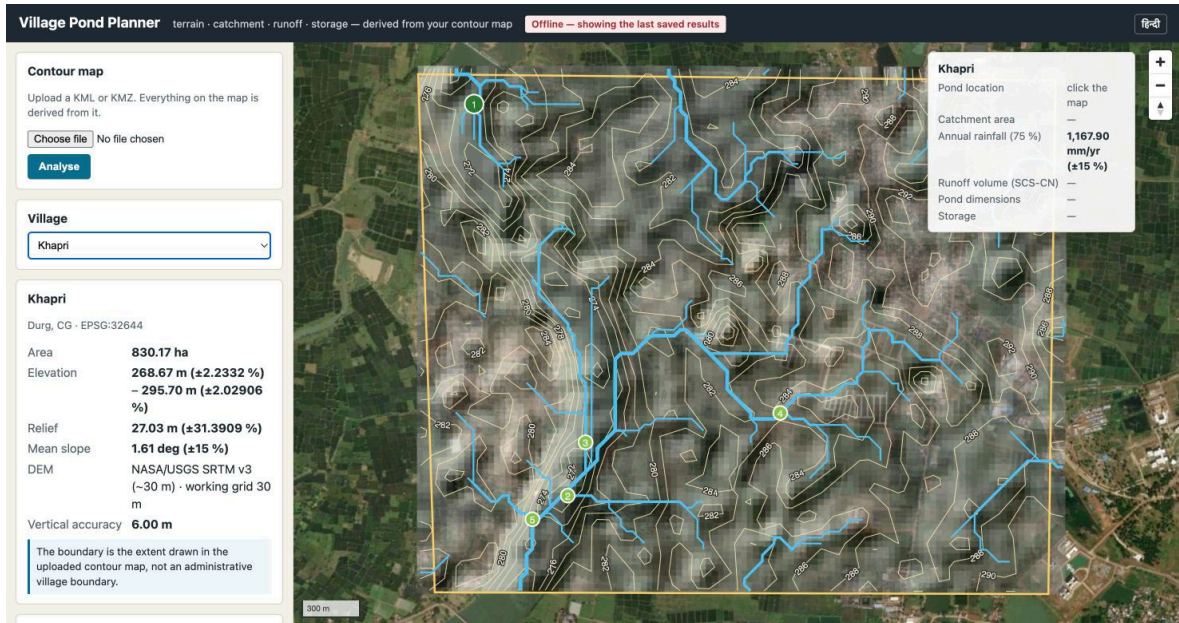
NDWI water mask: Otsu threshold, OpenCV morphology, connected components



All FR8 overlays on one map



Grafana dashboard during the load test



Offline: results served from cache with the API container stopped

**Coverage report: 86%**

coverage.py v7.15.4, created at 2026-08-26 14:42 +0530

| File                                   | statements | missing | excluded | coverage |
|----------------------------------------|------------|---------|----------|----------|
| app/engines/__init__.py                | 0          | 0       | 0        | 100%     |
| app/engines/design/__init__.py         | 0          | 0       | 0        | 100%     |
| app/engines/design/eav.py              | 48         | 0       | 0        | 100%     |
| app/engines/design/geometry.py         | 64         | 0       | 0        | 100%     |
| app/engines/design/optimiser.py        | 37         | 1       | 0        | 97%      |
| app/engines/design/spillway.py         | 19         | 0       | 0        | 100%     |
| app/engines/design/water_balance.py    | 59         | 1       | 0        | 98%      |
| app/engines/hydrology/__init__.py      | 0          | 0       | 0        | 100%     |
| app/engines/hydrology/catchment.py     | 90         | 2       | 0        | 98%      |
| app/engines/hydrology/conditioning.py  | 46         | 1       | 0        | 98%      |
| app/engines/hydrology/flow.py          | 75         | 1       | 0        | 99%      |
| app/engines/hydrology/siting.py        | 76         | 1       | 0        | 99%      |
| app/engines/hydrology/streams.py       | 58         | 0       | 0        | 100%     |
| app/engines/rainfall/__init__.py       | 0          | 0       | 0        | 100%     |
| app/engines/rainfall/service.py        | 38         | 5       | 0        | 87%      |
| app/engines/rainfall/statistics.py     | 80         | 4       | 0        | 95%      |
| app/engines/runoff/__init__.py         | 0          | 0       | 0        | 100%     |
| app/engines/runoff/curve_number.py     | 38         | 2       | 0        | 95%      |
| app/engines/runoff/methods.py          | 77         | 1       | 9        | 99%      |
| app/engines/suitability/__init__.py    | 0          | 0       | 0        | 100%     |
| app/engines/suitability/ahp.py         | 35         | 2       | 0        | 94%      |
| app/engines/suitability/constraints.py | 110        | 9       | 0        | 92%      |
| app/engines/suitability/water_mask.py  | 44         | 0       | 0        | 100%     |

Coverage report

# AI-based Village Pond Planning System 0.1.0 OAS 3.1

/openapi.json

Terrain, catchment and runoff analysis for village pond siting. Analysis routes are asynchronous: they return 202 with a job id to poll.

| health   |                                              |                                                  | ^ |
|----------|----------------------------------------------|--------------------------------------------------|---|
| GET      | /health                                      | Liveness probe                                   | ▼ |
| GET      | /ready                                       | Readiness probe                                  | ▼ |
| villages |                                              |                                                  | ^ |
| GET      | /api/v1/villages                             | List villages                                    | ▼ |
| POST     | /api/v1/villages                             | Create Village                                   | ▼ |
| GET      | /api/v1/villages/{village_id}                | Village detail                                   | ▼ |
| GET      | /api/v1/villages/{village_id}/summary        | Get Village Summary                              | ▼ |
| GET      | /api/v1/villages/{village_id}/imagery        | Get Village Imagery                              | ▼ |
| GET      | /api/v1/villages/{village_id}/siting         | Ranked pond sites from the last terrain analysis | ▼ |
| GET      | /api/v1/villages/{village_id}/available-land | Get Available Land                               | ▼ |
| POST     | /api/v1/villages/{village_id}/parcels:import | Import Parcels                                   | ▼ |

The deployed lab-VM instance: Swagger UI at <http://10.1.75.53:4269/docs>

 Village Pond Planner

How it worksWhat you getMethodologyValidationAPIReportOpen the planner

VILLAGE POND PLANNING · TERRAIN-FIRST

From a contour map to a  
costed pond — with the  
reasoning shown.

Upload a KML/KMZ of a village. Get the terrain, the streams, ranked pond sites, the catchment of any point you click, 45 years of rainfall, runoff by three methods and a pond design with fill reliability. Every number carries its unit and an honest uncertainty band.

Open the planner

See AOI 21.2517N 81.2970E, the sample village

✓ Validated against an independent model    ✓ Nothing hard-coded to one map

✓ Works offline after first load

Village Pond Planner

1. Stream

2. Pond

3. Pond catchment

4. Pond design

5. Pond reliability

6. Pond fill

7. Pond design

8. Pond reliability

9. Pond fill

10. Pond design

11. Pond reliability

12. Pond fill

13. Pond design

14. Pond reliability

15. Pond fill

16. Pond design

17. Pond reliability

18. Pond fill

19. Pond design

20. Pond reliability

21. Pond fill

22. Pond design

23. Pond reliability

24. Pond fill

25. Pond design

26. Pond reliability

27. Pond fill

28. Pond design

29. Pond reliability

30. Pond fill

31. Pond design

32. Pond reliability

33. Pond fill

34. Pond design

35. Pond reliability

36. Pond fill

37. Pond design

38. Pond reliability

39. Pond fill

40. Pond design

41. Pond reliability

42. Pond fill

43. Pond design

44. Pond reliability

45. Pond fill

46. Pond design

47. Pond reliability

48. Pond fill

49. Pond design

50. Pond reliability

51. Pond fill

52. Pond design

53. Pond reliability

54. Pond fill

55. Pond design

56. Pond reliability

57. Pond fill

58. Pond design

59. Pond reliability

60. Pond fill

61. Pond design

62. Pond reliability

63. Pond fill

64. Pond design

65. Pond reliability

66. Pond fill

67. Pond design

68. Pond reliability

69. Pond fill

70. Pond design

71. Pond reliability

72. Pond fill

73. Pond design

74. Pond reliability

75. Pond fill

76. Pond design

77. Pond reliability

78. Pond fill

79. Pond design

80. Pond reliability

81. Pond fill

82. Pond design

83. Pond reliability

84. Pond fill

85. Pond design

86. Pond reliability

87. Pond fill

88. Pond design

89. Pond reliability

90. Pond fill

91. Pond design

92. Pond reliability

93. Pond fill

94. Pond design

95. Pond reliability

96. Pond fill

97. Pond design

98. Pond reliability

99. Pond fill

100. Pond design

101. Pond reliability

102. Pond fill

103. Pond design

104. Pond reliability

105. Pond fill

106. Pond design

107. Pond reliability

108. Pond fill

109. Pond design

110. Pond reliability

111. Pond fill

112. Pond design

113. Pond reliability

114. Pond fill

115. Pond design

116. Pond reliability

117. Pond fill

118. Pond design

119. Pond reliability

120. Pond fill

121. Pond design

122. Pond reliability

123. Pond fill

124. Pond design

125. Pond reliability

126. Pond fill

127. Pond design

128. Pond reliability

129. Pond fill

130. Pond design

131. Pond reliability

132. Pond fill

133. Pond design

134. Pond reliability

135. Pond fill

136. Pond design

137. Pond reliability

138. Pond fill

139. Pond design

140. Pond reliability

141. Pond fill

142. Pond design

143. Pond reliability

144. Pond fill

145. Pond design

146. Pond reliability

147. Pond fill

148. Pond design

149. Pond reliability

150. Pond fill

151. Pond design

152. Pond reliability

153. Pond fill

154. Pond design

155. Pond reliability

156. Pond fill

157. Pond design

158. Pond reliability

159. Pond fill

160. Pond design

161. Pond reliability

162. Pond fill

163. Pond design

164. Pond reliability

165. Pond fill

166. Pond design

167. Pond reliability

168. Pond fill

169. Pond design

170. Pond reliability

171. Pond fill

172. Pond design

173. Pond reliability

174. Pond fill

175. Pond design

176. Pond reliability

177. Pond fill

178. Pond design

179. Pond reliability

180. Pond fill

181. Pond design

182. Pond reliability

183. Pond fill

184. Pond design

185. Pond reliability

186. Pond fill

187. Pond design

188. Pond reliability

189. Pond fill

190. Pond design

191. Pond reliability

192. Pond fill

193. Pond design

194. Pond reliability

195. Pond fill

196. Pond design

197. Pond reliability

198. Pond fill

199. Pond design

200. Pond reliability

201. Pond fill

202. Pond design

203. Pond reliability

204. Pond fill

205. Pond design

206. Pond reliability

207. Pond fill

208. Pond design

209. Pond reliability

210. Pond fill

211. Pond design

212. Pond reliability

213. Pond fill

214. Pond design

215. Pond reliability

216. Pond fill

217. Pond design

218. Pond reliability

219. Pond fill

220. Pond design

221. Pond reliability

222. Pond fill

223. Pond design

224. Pond reliability

225. Pond fill

226. Pond design

227. Pond reliability

228. Pond fill

229. Pond design

230. Pond reliability

231. Pond fill

232. Pond design

233. Pond reliability

234. Pond fill

235. Pond design

236. Pond reliability

237. Pond fill

238. Pond design

239. Pond reliability

240. Pond fill

241. Pond design

242. Pond reliability

243. Pond fill

244. Pond design

245. Pond reliability

246. Pond fill

247. Pond design

248. Pond reliability

249. Pond fill

250. Pond design

251. Pond reliability

252. Pond fill

253. Pond design

254. Pond reliability

255. Pond fill

256. Pond design

257. Pond reliability

258. Pond fill

259. Pond design

260. Pond reliability

261. Pond fill

262. Pond design

263. Pond reliability

264. Pond fill

265. Pond design

266. Pond reliability

267. Pond fill

268. Pond design

269. Pond reliability

270. Pond fill

271. Pond design

272. Pond reliability

273. Pond fill

274. Pond design

275. Pond reliability

276. Pond fill

277. Pond design

278. Pond reliability

279. Pond fill

280. Pond design

281. Pond reliability

282. Pond fill

283. Pond design

284. Pond reliability

285. Pond fill

286. Pond design

287. Pond reliability

288. Pond fill

289. Pond design

290. Pond reliability

291. Pond fill

292. Pond design

293. Pond reliability

294. Pond fill

295. Pond design

296. Pond reliability

297. Pond fill

298. Pond design

299. Pond reliability

300. Pond fill

301. Pond design

302. Pond reliability

303. Pond fill

304. Pond design

305. Pond reliability

306. Pond fill

307. Pond design

308. Pond reliability

309. Pond fill

310. Pond design

311. Pond reliability

312. Pond fill

313. Pond design

314. Pond reliability

315. Pond fill

316. Pond design

317. Pond reliability

318. Pond fill

319. Pond design

320. Pond reliability

321. Pond fill

322. Pond design

323. Pond reliability

324. Pond fill

325. Pond design

326. Pond reliability

327. Pond fill

328. Pond design

329. Pond reliability

330. Pond fill

331. Pond design

332. Pond reliability

333. Pond fill

334. Pond design

335. Pond reliability

336. Pond fill

337. Pond design

338. Pond reliability

339. Pond fill

340. Pond design

341. Pond reliability

342. Pond fill

343. Pond design

344. Pond reliability

345. Pond fill

346. Pond design

347. Pond reliability

348. Pond fill

349. Pond design

350. Pond reliability

351. Pond fill

352. Pond design

353. Pond reliability

354. Pond fill

355. Pond design

356. Pond reliability

357. Pond fill

358. Pond design

359. Pond reliability

360. Pond fill

361. Pond design

362. Pond reliability

363. Pond fill

364. Pond design

365. Pond reliability

366. Pond fill

367. Pond design

368. Pond reliability

369. Pond fill

370. Pond design

371. Pond reliability

372. Pond fill

373. Pond design

374. Pond reliability

375. Pond fill

376. Pond design

377. Pond reliability

378. Pond fill

379. Pond design

380. Pond reliability

381. Pond fill

382. Pond design

383. Pond reliability

384. Pond fill

385. Pond design

386. Pond reliability

387. Pond fill

388. Pond design

389. Pond reliability

390. Pond fill

391. Pond design

392. Pond reliability

393. Pond fill

394. Pond design

395. Pond reliability

396. Pond fill

397. Pond design

398. Pond reliability

399. Pond fill

400. Pond design

401. Pond reliability

402. Pond fill

403. Pond design

404. Pond reliability

405. Pond fill

406. Pond design

407. Pond reliability

408. Pond fill

409. Pond design

410. Pond reliability

411. Pond fill

412. Pond design

413. Pond reliability

414. Pond fill

415. Pond design

416. Pond reliability

417. Pond fill

418. Pond design

419. Pond reliability

420. Pond fill

421. Pond design

422. Pond reliability

423. Pond fill

424. Pond design

425. Pond reliability

426. Pond fill

427. Pond design

428. Pond reliability

429. Pond fill

430. Pond design

431. Pond reliability

432. Pond fill

433. Pond design

434. Pond reliability

435. Pond fill

436. Pond design

437. Pond reliability

438. Pond fill

439. Pond design

440. Pond reliability

441. Pond fill

442. Pond design

443. Pond reliability

444. Pond fill

445. Pond design

446. Pond reliability

447. Pond fill

448. Pond design

449. Pond reliability

450. Pond fill

451. Pond design

452. Pond reliability

453. Pond fill

454. Pond design

455. Pond reliability

456. Pond fill

457. Pond design

458. Pond reliability

459. Pond fill

460. Pond design

461. Pond reliability

462. Pond fill

463. Pond design

464. Pond reliability

465. Pond fill

466. Pond design

467. Pond reliability

468. Pond fill

469. Pond design

470. Pond reliability

471. Pond fill

472. Pond design

473. Pond reliability

474. Pond fill

475. Pond design

476. Pond reliability

477. Pond fill

478. Pond design

479. Pond reliability

480. Pond fill

481. Pond design

482. Pond reliability

483. Pond fill

484. Pond design

485. Pond reliability

486. Pond fill

487. Pond design

488. Pond reliability

489. Pond fill

490. Pond design

491. Pond reliability

492. Pond fill

493. Pond design

494. Pond reliability

495. Pond fill

496. Pond design

497. Pond reliability

498. Pond fill

499. Pond design

500. Pond reliability

501. Pond fill

502. Pond design

503. Pond reliability

504. Pond fill

505. Pond design

506. Pond reliability

507. Pond fill

508. Pond design

509. Pond reliability

510. Pond fill

511. Pond design

512. Pond reliability

513. Pond fill

514. Pond design

515. Pond reliability

516. Pond fill

517. Pond design

518. Pond reliability

519. Pond fill

520. Pond design

521. Pond reliability

522. Pond fill

523. Pond design

524. Pond reliability

525. Pond fill

526. Pond design

527. Pond reliability

528. Pond fill

529. Pond design

530. Pond reliability

531. Pond fill

532. Pond design

533. Pond reliability

534. Pond fill

535. Pond design

536. Pond reliability

537. Pond fill

538. Pond design

539. Pond reliability

540. Pond fill

541. Pond design

542. Pond reliability

543. Pond fill

544. Pond design

545. Pond reliability

546. Pond fill

547. Pond design

548. Pond reliability

549. Pond fill

550. Pond design

551. Pond reliability

552. Pond fill

553. Pond design

554. Pond reliability

555. Pond fill

556. Pond design

557. Pond reliability

558. Pond fill

559. Pond design

560. Pond reliability

561. Pond fill

562. Pond design

563. Pond reliability

564. Pond fill

565. Pond design

566. Pond reliability

567. Pond fill

568. Pond design

569. Pond reliability

569. Pond fill

RAINFALL RECORD  
45 years  
ERAS-Land, daily

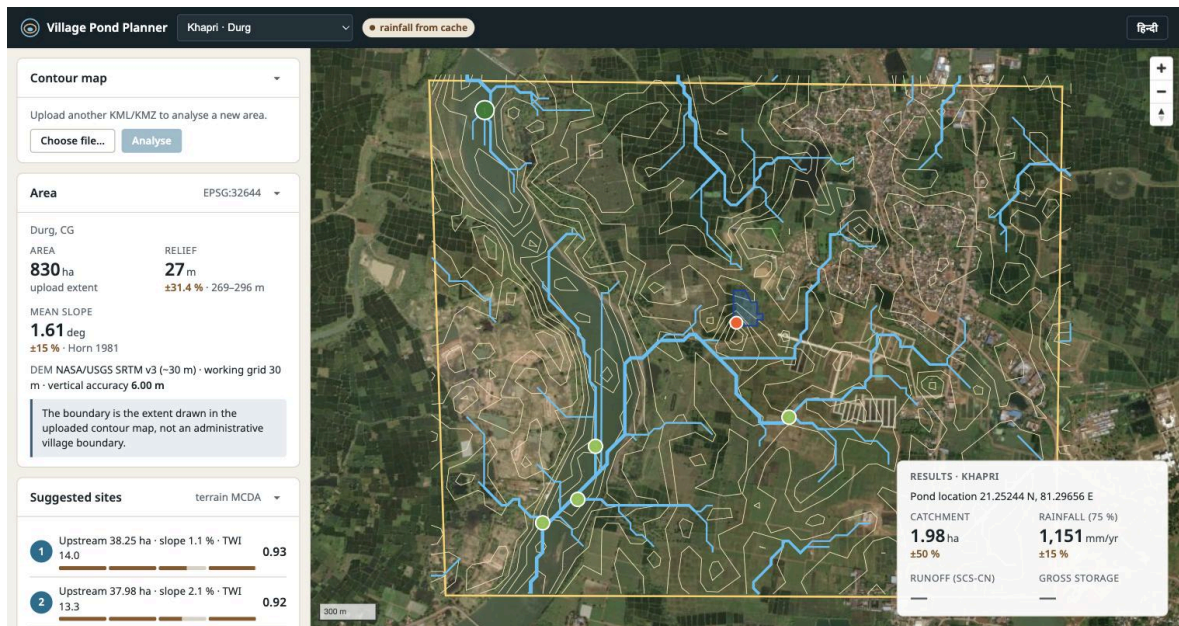
CATCHMENT VS PYSHEDS  
2-3%  
on the main outlets

RUNOFF METHODS  
3  
SCS-CN - rational - Strange

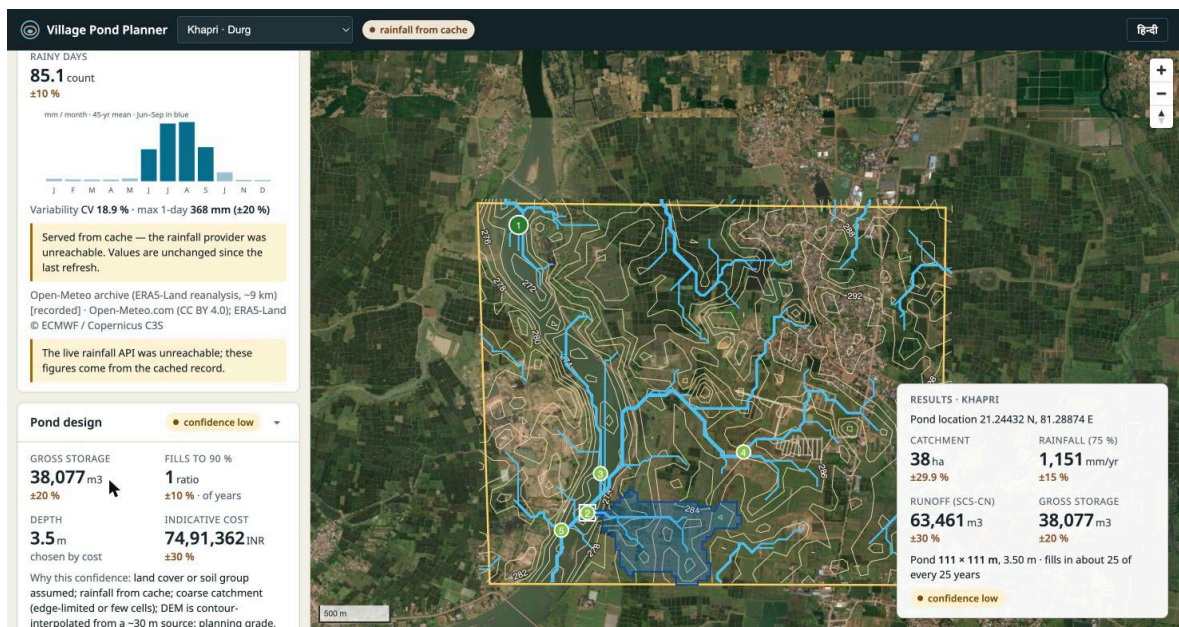
MAP LAYERS  
13  
all on one map

AREAS ANALYSED HERE  
1  
AOI 21.2517N 81.2970E

The landing page served from the lab VM



The planner on the lab VM: click-to-catchment on the analysed sample — snapped outlet, catchment polygon, ranked sites



The full chain from the deployed UI: catchment 38 ha → runoff 63,461 m<sup>3</sup> → 38,077 m<sup>3</sup> pond, 111 × 111 m × 3.5 m, with its confidence label